Lab #4b:
CLIENT-SIDE SCRIPTING (JSON and jQuery)

Topic	**Web page development using JSON, modern JavaScript, and jQuery**
Domain of Learning	Psychomotor (P2: Set; P3: Guided Response; P4: Mechanism)
Learning Objective	Retrieve, parse, access, and display JSON data using modern JavaScript; compare native DOM operations with jQuery.
Lab Activity Objective	Create a valid JSON dataset, load and display selected data using fetch(), and implement equivalent DOM updates and events using jQuery.

Instructions

a. Complete Questions 1–3 individually.
b. Use a code editor, a modern browser, and a local development server (e.g., VS Code Live Server) to test fetch() with local JSON files. Do not open the HTML directly using file:// for Question 2.
c. Use separate files named books.json, books.html, and jquery-dom.html. Push your work to a GitHub repository, including a README.md with setup instructions and screenshots.
d. Submit a concise PDF report through AUTHOR containing the required outputs, brief explanations, and your GitHub repository URL.

Question 1 — Create and Structure JSON Data (5 marks)

Consider the following collection of books:

Property	**Book 1**	**Book 2**
title	The Great Gatsby	To Kill a Mockingbird
author	F. Scott Fitzgerald	Harper Lee
publicationYear	1925	1960
genres	Fiction, Classics	Fiction, Coming-of-Age

Create a valid file named books.json containing a top-level "books" array with two book objects. Each book must include title (string), author (string), publicationYear (number), and genres (array of strings). Ensure the JSON uses double-quoted property names and has no trailing commas.

Expected evidence:

A screenshot of the complete JSON file and confirmation that it parses successfully.



Question 2 — Retrieve and Display JSON Data (5 marks)

Create books.html. Using the native JavaScript fetch() API, read books.json and display the following items within the webpage (not only in the browser console):

(a) Title of the first book. (b) Author of the second book. (c) Genres of both books, clearly grouped by book title.

Use textContent and/or createElement() to display text safely. Handle unsuccessful requests or invalid JSON by displaying a user-friendly error message. You may use .then() or async/await. Run the HTML through a local server and capture the browser output.

Expected output:
The Great Gatsby; Harper Lee; The Great Gatsby — Fiction, Classics; To Kill a Mockingbird — Fiction, Coming-of-Age.

books.json | books.html

Lab4b Assignment > books.html > html > body > script > then() callback

```
<!DOCTYPE html>
<html>
<head>
    <title>Books Information</title>
</head>
<body>

    <h2>Books Information</h2>

    <div id="output"></div>

    <script>
        fetch("books.json")
            .then(response => {
                if (!response.ok) {
                    throw new Error("Unable to load JSON file.");
                }
                return response.json();
            })
            .then(data => {

                const output = document.getElementById("output");

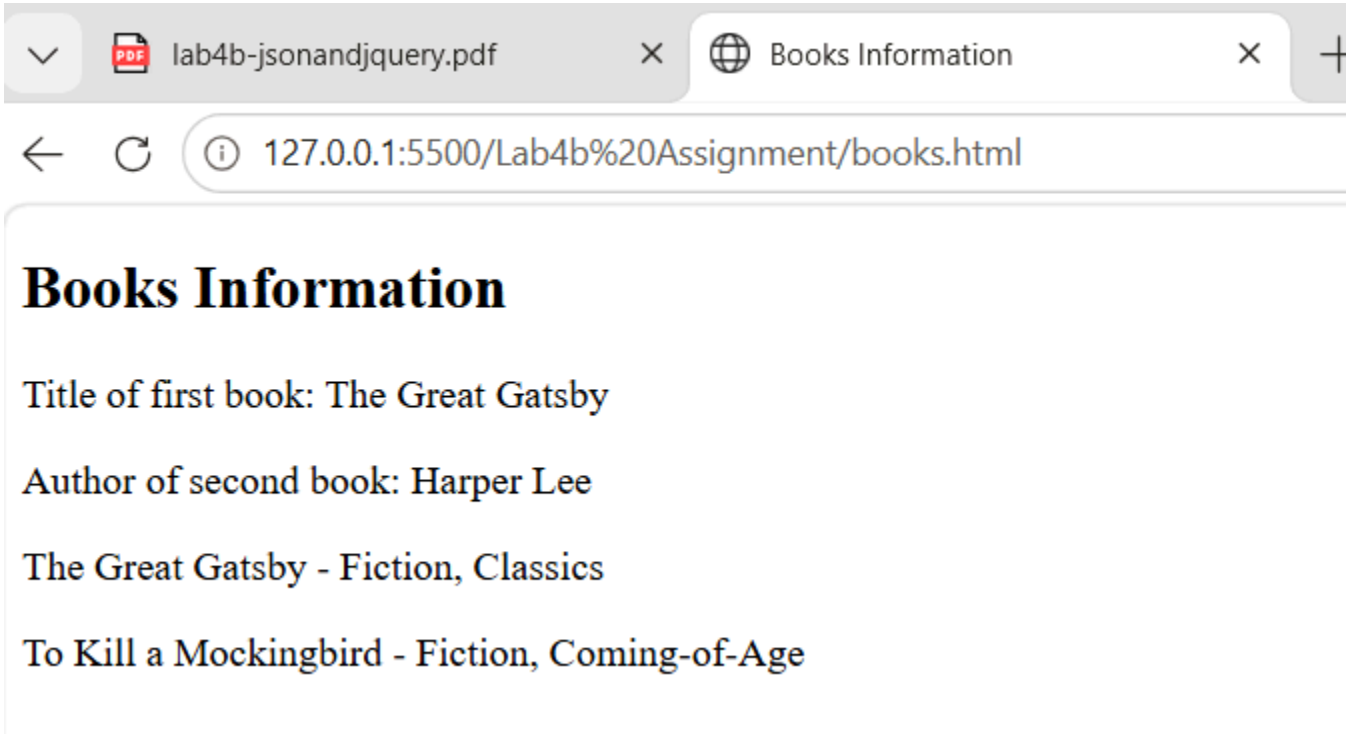
                const p1 = document.createElement("p");
                p1.textContent = "Title of first book: " + data.books[0].title;

                const p2 = document.createElement("p");
                p2.textContent = "Author of second book: " + data.books[1].author;

                const p3 = document.createElement("p");
                p3.textContent =
                    data.books[0].title + " - " +
                    data.books[0].genres.join(", ");

                const p4 = document.createElement("p");
                p4.textContent =
```

```
35                  const p4 = document.createElement("p");
36                  p4.textContent =
37                      data.books[1].title + " - " +
38                      data.books[1].genres.join(", ");
39
40                  output.appendChild(p1);
41                  output.appendChild(p2);
42                  output.appendChild(p3);
43                  output.appendChild(p4);
44              })
45              .catch(error => {
46                  document.getElementById("output").textContent =
47                      "Error: " + error.message;
48              });
49      </script>
50
51  </body>
52  </html>
```



Question 3 — Compare Native JavaScript and jQuery (5 marks)

Create jquery-dom.html. Include jQuery 4.0.0 using the official full-build CDN link below (not the slim build):

https://code.jquery.com/jquery-4.0.0.min.js

Use the following HTML elements and CSS class:

```
<div id="myElement">Original Text</div>

<div id="myDIV"><p>I am a myDIV.</p></div>

<button id="myButton">Click me</button>

.myStyle { background-color: coral; padding: 16px; }
```

Write the jQuery equivalents for all three native JavaScript operations below. The result must (a) replace the text of #myElement with "Modified Text", (b) add the myStyle class to #myDIV, and (c) respond to a click on #myButton by showing "Button clicked!".

```
// (a) document.getElementById("myElement").textContent = "Modified Text";

// (b) document.querySelector("#myDIV").classList.add("myStyle");

// (c) document.getElementById("myButton").addEventListener("click", () =>
alert("Button clicked!"));
```

Use .text(), .addClass(), and .on("click", ...). In your PDF report, briefly state one advantage of native JavaScript and one situation where an existing project may still use jQuery.

```html
<!DOCTYPE html>
<html>
<head>
    <title>jQuery DOM Example</title>

    <script src="https://code.jquery.com/jquery-4.0.0.min.js"></script>

    <style>
        .myStyle {
            background-color: coral;
            padding: 16px;
        }
    </style>
</head>
<body>

    <div id="myElement">Original Text</div>

    <br>

    <div id="myDIV">
        <p>I am a myDIV.</p>
    </div>

    <br>

    <button id="myButton">Click me</button>

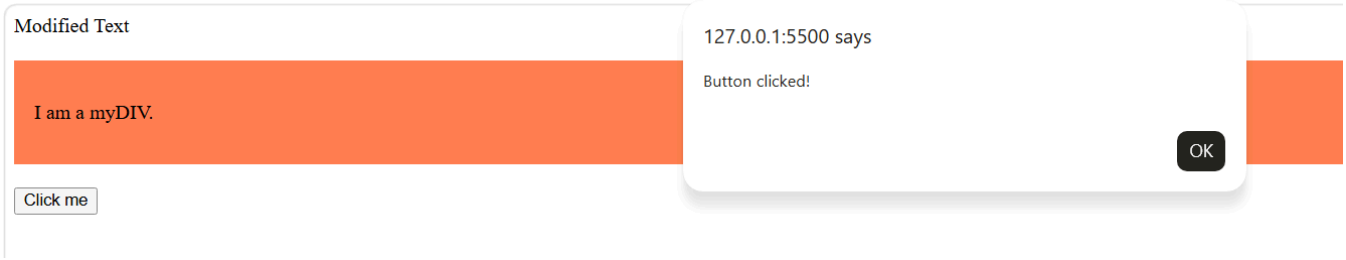
    <script>

        $("#myElement").text("Modified Text");

        $("#myDIV").addClass("myStyle");

        $("#myButton").on("click", function () {
            alert("Button clicked!");
```

```
36              alert("Button clicked!");
37          });
38
39      </script>
40
41  </body>
42  </html>
```



Advantage of Native JavaScript:

Native JavaScript does not require any external library, making web pages load faster and reducing dependencies.

Why Some Projects Still Use jQuery:

Many older websites were built using jQuery. Developers may continue using it to maintain compatibility and avoid rewriting existing code.

Submission Checklist

- books.json and books.html work through a local server
- jquery-dom.html works correctly
- GitHub repository and README.md included □ PDF contains screenshots, explanations, and repository link

Assessment:

Assessment A — JSON Data Handling and jQuery Integration (CLO2 — Psychomotor)

Scoring: For each criterion, select a performance level (0–5) and calculate weighted marks as (score ÷ 5) × 5. Total: 15 marks.

Criteria	**Excellent (5)**	**Good (4)**	**Satisfactory (3)**	**Needs Improvement (1–2)**	**Unsatisfactory (0)**	**Marks**
Parsing JSON Data (P4)	Valid, correctly typed JSON with an accurate books array and genre arrays.	Valid JSON with minor structure or typing issues.	Mostly valid structure; some data types or fields incorrect.	Incomplete JSON or significant structural errors requiring guidance.	No usable JSON dataset.	___ /5 × 5
Manipulating JSON Data (P4)	Successfully fetches JSON and correctly displays all required data with error handling.	Retrieves and displays all items with minor presentation or error-handling issues.	Retrieves JSON and displays some required data correctly.	Partial retrieval or incorrect item selection; substantial assistance needed.	No functional JSON retrieval or display.	___ /5 × 5
jQuery Integration for DOM and Events (P4)	All three operations work correctly using appropriate jQuery methods.	All three mostly work with minor implementation issues.	Two of three operations work correctly.	Only one operation works or methods are used incorrectly.	No functional jQuery operations.	___ /5 × 5

CLO2 Total: _____ /15

Assessment B — Presentation and Understanding (CLO3 — Affective)
Scoring: Clarity and Confidence = (score ÷ 5) × 2; Explanation = (score ÷ 5) × 3. Total: 5 marks.

Criteria	**Excellent (5)**	**Good (4)**	**Satisfactory (3)**	**Needs Improvement (1–2)**	**Unsatisfactory (0)**	**Marks**
Clarity and Confidence (A1)	Demonstrates outputs clearly, confidently, and independently.	Clear demonstration with minor hesitation.	Adequate demonstration with some uncertainty.	Unclear demonstration needing repeated prompting.	Unable to demonstrate the work.	___ /5 × 2
Explanation of JSON, JavaScript and jQuery (A2)	Explains JSON structure, fetch(), and jQuery operations accurately, with reasons for choices.	Explains most concepts correctly with minor gaps.	Provides a basic explanation of key concepts.	Limited explanation and cannot justify several choices.	Unable to explain relevant concepts.	___ /5 × 3

CLO3 Total: _____ /5 **OVERALL: _____ /20**

Lecturer note: The original rubric criteria and weightings are retained. Descriptors are updated to match the revised Questions 1–3. Verify P/A levels against the approved course CLO mapping before use.